



Flood Polynomial Model — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 29-March-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

1 Test Results — Flood Polynomial Model (MKM-FPO-001)

Tests run on **29 March 2026 at 10:29:14**.

Table 1: Test Results Summary — Flood Polynomial Model (MKM-FPO-001)

Total Tests	33
Passed	33
Failed	0
Skipped	0
Duration	0.00s

Table 2: Detailed Test Results — Flood Polynomial Model

Test Description	Acceptance Criteria	Result	Time
Coefficients used by model match config/models.py.	<code>hasattr(flood_poly_coeffs, 'a');</code> <code>hasattr(flood_poly_coeffs, 'f')</code> (+1 more)	Pass	0.000s
Fit metadata from config	<code>flood_poly_fit["model_id"] == "flood-poly-v1";</code> <code>flood_poly_fit["r_squared"] > 0.9</code>	Pass	0.000s
Log eps from config	<code>flood_poly_log_eps == 1e-8</code>	Pass	0.000s
Storm hours from config	<code>flood_poly_storm_hours == 168</code>	Pass	0.000s
Coefficients count	<code>card["num.parameters"] == 6</code>	Pass	0.000s
Fit quality has metrics	<code>"r_squared" in fq;</code> <code>"mae" in fq</code> (+1 more)	Pass	0.000s
Has required fields	<code>"model_id" in card;</code> <code>"version" in card</code> (+3 more)	Pass	0.000s
Model id matches config	<code>card["model_id"] == flood_poly_fit["model_id"]</code>	Pass	0.000s
Returns dict	<code>isinstance(card, dict)</code>	Pass	0.000s
Water far above severe at late hour should be near 1.0.	<code>p > 0.9</code>	Pass	0.000s
Water well above severe threshold should give high P(flood).	<code>p > 0.8</code>	Pass	0.000s
hour=167 (last storm hour) should not raise.	<code>0.0 <= p <= 1.0</code>	Pass	0.000s
hour=0 should not raise.	<code>0.0 <= p <= 1.0</code>	Pass	0.000s
P(flood) at hour 0 should be lower than mid-storm for moderate water.	<code>p_early < p_mid</code>	Pass	0.000s
In range	<code>0.0 <= result <= 1.0;</code> <code>0.0 <= result <= 1.0</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Water well below severe threshold should give low P(flood).	<code>p < 0.1</code>	Pass	0.000s
Higher water level should give higher P(flood) at same hour.	<code>p_low < p_mid < p_high</code>	Pass	0.000s
Negative severe returns zero	<code>p_flood.poly(water_level=5.0, hour=48, severe_level=-1.0) ...;</code> <code>p_flood.simple(5.0, -1.0) == 0.0</code>	Pass	0.000s
Returns float	<code>isinstance(result, float);</code> <code>isinstance(result, float)</code>	Pass	0.000s
Near-zero water level should give near-zero P(flood).	<code>p < 0.01</code>	Pass	0.000s
Zero severe returns zero	<code>p_flood.poly(water_level=5.0, hour=48, severe_level=0.0) ...;</code> <code>p_flood.simple(5.0, 0.0) == 0.0</code>	Pass	0.000s
All in range	<code>all(0.0 <= p <= 1.0 for p in result)</code>	Pass	0.000s
Correct length	<code>len(result) == 5</code>	Pass	0.000s
Empty list	<code>p_flood.series([], severe_level=6.58)</code> <code>== []</code>	Pass	0.000s
Returns list	<code>isinstance(result, list)</code>	Pass	0.000s
Above severe capped at one	<code>p_flood.simple(10.0, 6.58) == 1.0</code>	Pass	0.000s
At severe equals one	<code>p_flood.simple(6.58, 6.58) == 1.0</code>	Pass	0.000s
In range	<code>0.0 <= result <= 1.0; 0.0 <= result <= 1.0</code>	Pass	0.000s
Negative severe returns zero	<code>p_flood.poly(water_level=5.0, hour=48, severe_level=-1.0) ...;</code> <code>p_flood.simple(5.0, -1.0) == 0.0</code>	Pass	0.000s
$P = \min(1, w/s)$.	<code>abs(p_flood.simple(3.29, 6.58) - 0.5) < 0.01</code>	Pass	0.000s
Returns float	<code>isinstance(result, float);</code> <code>isinstance(result, float)</code>	Pass	0.000s
Zero severe returns zero	<code>p_flood.poly(water_level=5.0, hour=48, severe_level=0.0) ...;</code> <code>p_flood.simple(5.0, 0.0) == 0.0</code>	Pass	0.000s
Zero water	<code>p_flood.simple(0.0, 6.58) == 0.0</code>	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
29-Mar-2026	1.0	Initial test suite (33 tests)	D. Kelly